

Digital Image Processing Assignment 02 – Report Analysis

Name : Naila Syakira

NIM : 25/555286/PA/23270

In this experiment, an image enhancement toolkit was built to improve three common types of degraded images: dark (underexposed) images, blurred images, and low-contrast images. The goal was not just to write working code, but to understand why each technique works and how its parameters affect the final result.

The method used was straightforward. One clean portrait image was taken as the reference, and three degraded versions were created from it: a dark version, a blurred version, and a low-contrast version. Each degraded version was then processed by a dedicated enhancement function, and the results were compared side by side against both the degraded input and the original clean image.

The final notebook contains five main functions plus a helper.

1. The *show_images* helper displays a list of images side by side with titles. It handles both colour and grayscale input and is used for every comparison in the notebook.

```
def show_images(titles, images, figsize=(15, 5), cmap='gray'):
    plt.figure(figsize=figsize)
    for i, (title, img) in enumerate(zip(titles, images)):
        plt.subplot(1, len(images), i + 1)
        if len(img.shape)==2:
            plt.imshow(img, cmap=cmap, vmin=0, vmax=255)
        else:
            plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        plt.title(title)
        plt.axis('off')
    plt.show()
plt.tight_layout()
plt.show()
```

2. *enhance_dark_image* converts the image to LAB colour space, applies CLAHE to the L channel only, then applies gamma correction with $\gamma = 1.5$ to lift the shadows. Finally, it boosts saturation to counteract the desaturation that gamma causes. The result is a noticeably brighter and more colourful image from a dark input.

```
def enhance_dark_image(img, clup_limit=3.0, title_grid=(4,4), gamma=1.5,
    saturation=1.9):
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)

    clahe = cv2.createCLAHE(clipLimit=clup_limit, tileGridSize=title_grid)
    l_clahe = clahe.apply(l)
    lab_clahe = cv2.merge((l_clahe, a, b))
```

```

img_clahe = cv2.cvtColor(lab_clahe, cv2.COLOR_LAB2BGR)

inv_gamma = 1.0 / gamma
table = np.array([(i / 255.0) ** inv_gamma) * 255 for i in np.arange(0,
256)]).astype("uint8")
img_gamma = cv2.LUT(img_clahe, table)

if saturation != 1:
    hsv = cv2.cvtColor(img_gamma, cv2.COLOR_BGR2HSV).astype(np.float32)
    hsv[:, :, 1] = np.clip(hsv[:, :, 1] * saturation, 0, 255)
    img_gamma = cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2BGR)

return img_gamma

```

3. For low-contrast images, two versions were implemented. The simpler one (*enhance_low_contrast*) applies CLAHE directly. The better one (*enhance_low_contrast_v2*) first does a percentile-based contrast stretch on each channel (*enhance_low_contrast_stretch*), then applies CLAHE on top. The stretch-then-CLAHE ordering gives a much richer result, because the stretch expands the histogram globally and CLAHE then enhances local detail. This two-stage approach was the clearest improvement made to the original pipeline.

```

def enhance_low_contrast(img, clip_limit=4.0, tile_grid=(8, 8)):
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid)
    l = clahe.apply(l)
    return cv2.cvtColor(cv2.merge((l, a, b)), cv2.COLOR_LAB2BGR)

def enhance_low_contrast_stretch(img, low_pct=2, high_pct=98):
    out = np.zeros_like(img)
    for c in range(3):
        ch = img[:, :, c]
        lo, hi = np.percentile(ch, (low_pct, high_pct))
        if hi > lo:
            stretched = (ch.astype(np.float32) - lo) * (255.0 / (hi - lo))
            out[:, :, c] = np.clip(stretched, 0, 255).astype(np.uint8)
        else:
            out[:, :, c] = ch
    return out

def enhance_low_contrast_v2(img, low_pct=2, high_pct=98, clip_limit=3.0, tile_grid=(8,
8)):
    stretched = enhance_low_contrast_stretch(img, low_pct, high_pct)
    lab = cv2.cvtColor(stretched, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid)
    l = clahe.apply(l)
    return cv2.cvtColor(cv2.merge((l, a, b)), cv2.COLOR_LAB2BGR)

```

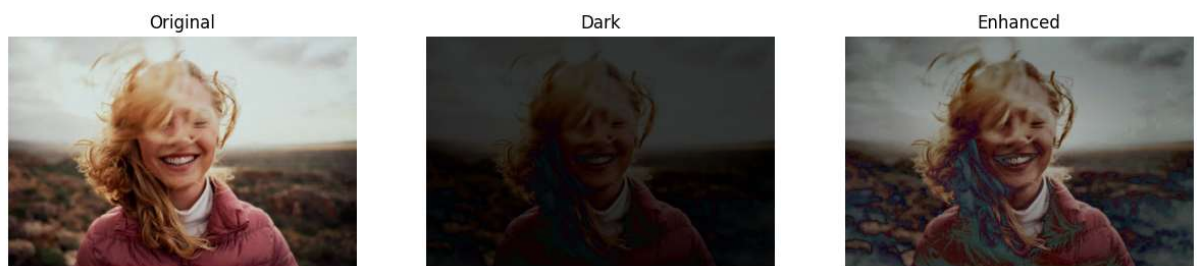
4. *enhance_blurred_image* implements unsharp masking in three lines. It blurs the image with a Gaussian, then combines the original and blurred versions using the correct weights. Using a (0, 0) kernel with an explicit sigma value avoids the odd-kernel pitfall encountered earlier.

```
def enhance_blurred_image(img, radius=3, amount=1.8):  
    blurred = cv2.GaussianBlur(img, (0, 0), radius)  
    return cv2.addWeighted(img, 1 + amount, blurred, -amount, 0)
```

The demo section loads one portrait, creates the three degraded versions from it, runs each enhancer, and displays three side-by-side comparisons. The enhanced results are saved to the (images/output/) folder for the report.

1. Enhanced Dark Image

The dark input was created by applying `cv2.convertScaleAbs(original, alpha=0.30, beta=-15)`, which compresses almost the entire image into the darkest quarter of the histogram. The subject's face, hair, and jacket merge into the background, the landscape collapses into muddy dark tones, and colour is nearly lost.



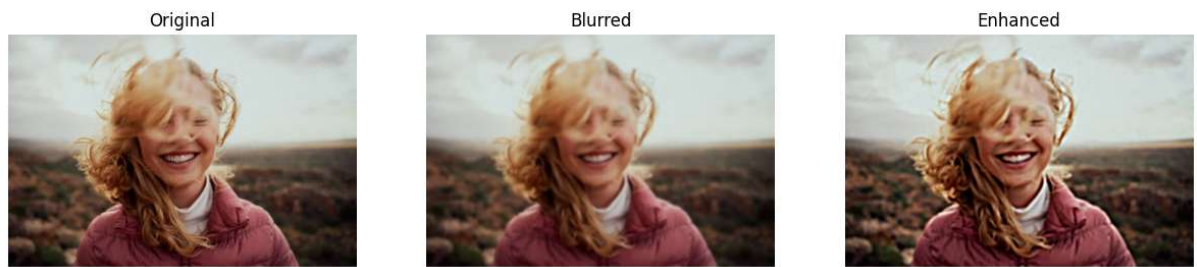
After running it through `enhance_dark_image`, the output is dramatically brighter: the face, hair, and jacket are clearly visible, the background landscape has returned, and the overall structure of the original is recovered from the shadows. However, the result is not a clean recovery as the subject's blonde hair has picked up visible blue and purple streaks, the landscape shows a strong blue-green cast, and smooth regions look slightly grainy and patchy. These artefacts come from the LAB a/b colour boost amplifying tiny colour deviations in the dark input, and from CLAHE and gamma amplifying compression noise that was invisible in the darker version.

Compared to the original, the enhanced image is much closer in brightness and structure than the dark input, but it trades correct colour and clean tonality for brightness. This shows that enhancement is not recovery: aggressive parameters can fix one problem (darkness) while introducing another (colour distortion and noise), and a gentler colour boost with a light pre-denoise step would likely give a more natural result.

2. Enhanced Blurred Image

The input is the portrait after Gaussian blurring with a 7×7 kernel. Fine detail such as individual hair strands, eyelashes, and fabric texture has been smoothed away, and edges look

soft and undefined.



The overall structure of the image is still recognizable, but it looks out of focus. The enhanced output restores much of the edge definition, so hair strands and clothing folds once again have visible boundaries. However, the recovery is not perfect, unsharp masking can only amplify edge information that still exists, so some of the very fine detail lost to the blur remains unrecoverable. The result is a middle ground: sharp enough to look in-focus, but not identical to the original.

3. Enhanced Low Contrast Image

The input is a flattened version of the portrait, created by compressing pixel values with $\times 0.25 + 100$. The entire image sits within a narrow band of mid-grey values, giving it a washed-out, hazy appearance. Dark regions are not truly dark, bright regions are not truly bright, and colour feels muted.



The enhanced output shows a dramatic improvement: the dark tones have been pulled down toward black, the bright tones pushed up toward white, and the mid-tones redistributed. The result has strong global contrast and better-defined local detail, thanks to the CLAHE step after the stretch. Skin tones look richer, the background separates cleanly from the subject, and the image has a "punch" that the input completely lacked.